

Python 期末复习笔记

1. 基础输入输出

```
# =====
# print 输出函数 -- 把值打印到控制台
# =====
print("hello world")          # 输出: hello world -- 直接打印字符串
print("hello" + "world")      # 输出: helloworld -- 字符串用 + 号拼接
print(1, 2, 3, sep="-")       # 输出: 1-2-3 -- sep 参数设置多个参数之间的分隔符
print("hello", end=" ")       # 输出: hello (后面是空格而不是换行) -- end 参数
                                # 设置打印结束符, 默认是换行 \n
print()                       # 输出空行 -- 不带参数就打印一个换行

# =====
# input 输入函数 -- 从键盘读取用户输入
# =====
name = input("请输入名字: ")   # input() 返回的永远是字符串 string 类型
age = int(input("请输入年龄: ")) # 用 int() 把字符串转成整数后再接收

# =====
# 转义字符 -- 用反斜杠 \ 开头的特殊字符
# =====
print('let's go!')            # 输出: let's go! -- \' 表示单引号本身
print("第一行\n第二行")       # 输出: 第一行 / 第二行 -- \n 表示换行
print("姓名\t年龄\t城市")      # 输出: 姓名    年龄    城市 -- \t 表示制表符(对齐用)
print("D:\\Python\\test.py")  # 输出: D:\Python\test.py -- \\ 表示反斜杠本身
print("""多行
文本""")                      # 三引号可以直接写多行字符串, 保留换行格式
```

2. 变量命名

```
# =====
# 命名规则 (5 条必须记住)
# =====
# 1. 只能包含字母、数字和下划线 _
# 2. 不能以数字开头
# 3. 不能使用 Python 保留字 (如 if、for、while、class 等)
# 4. 区分大小写 (Name 和 name 是两个不同变量)
# 5. 应具有描述性 (见名知意)

user_name = "Alice"           # 下划线命名法 snake_case -- Python 推荐风格
userName = "Bob"              # 驼峰命名法 camelCase -- 其他语言常用
MAX_SIZE = 100                # 全大写表示常量 -- 只是约定, Python 没有真正的常量

# =====
# 多变量赋值
# =====
a, b, c = 1, 2, 3             # 一行给多个变量赋值, a=1, b=2, c=3
```

```

x = y = z = 0                # 多个变量赋相同的值，x=y=z=0

# =====
# 变量交换 —— Python 特色，不需要临时变量
# =====
a, b = 10, 20
a, b = b, a                    # 交换后 a=20, b=10 —— Python 独有的语法糖

# =====
# 删除变量
# =====
del a                          # 删除变量 a，释放内存空间

```

3. 数据类型与转换

```

# =====
# 整数型 int —— 没有大小限制，可以无限大
# =====
a = 10                        # 十进制整数 10
b = -5                        # 负整数 -5
c = 0b1010                    # 二进制 0b 开头 → 十进制 10
d = 0o12                       # 八进制 0o 开头 → 十进制 10
e = 0xA                       # 十六进制 0x 开头 → 十进制 10
print(a)                      # 输出：10

# =====
# 浮点型 float —— 带小数点的数
# =====
pi = 3.14159                  # 普通浮点数
f = 2.5e2                      # 科学计数法 →  $2.5 \times 10^2 = 250.0$ 
print(pi)                     # 输出：3.14159

# =====
# 复数型 complex —— 实部 + 虚部j
# =====
c = 3 + 4j                    # 实部 3，虚部 4
print(c.real)                 # 输出：3.0 —— 取实部
print(c.imag)                 # 输出：4.0 —— 取虚部

# =====
# 布尔型 bool —— 只有 True 和 False 两个值
# =====
flag = True                   # 布尔值 True（首字母大写）
print(True and False)        # 输出：False —— 与运算，两真才真
print(True or False)         # 输出：True —— 或运算，一真就真
print(not True)               # 输出：False —— 非运算，取反

# =====
# 空值 None —— 表示什么都没有
# =====
result = None                 # None 是一个特殊的空值
print(result is None)         # 输出：True —— 判断是否为 None 用 is，不要用 ==

# =====
# 查看数据类型 —— type() 函数

```

```

# =====
print(type(10))          # 输出: <class 'int'> -- 整数类型
print(type(3.14))        # 输出: <class 'float'> -- 浮点类型
print(type("hello"))     # 输出: <class 'str'> -- 字符串类型
print(type(True))        # 输出: <class 'bool'> -- 布尔类型

# =====
# 自动类型转换 -- 不同类型运算时自动升级
# =====
result = 10 + 3.5         # int + float → 自动转成 float
print(result)            # 输出: 13.5 -- 结果是浮点数

# =====
# 强制类型转换 -- 主动把一种类型转成另一种
# =====
print(int(3.7))          # 输出: 3 -- float 转 int, 直接截断小数 (不是四舍五入)
print(float(5))          # 输出: 5.0 -- int 转 float, 加个小数点
print(str(100))          # 输出: "100" -- 转成字符串
print(bool(0))           # 输出: False -- 0 转布尔是 False
print(bool(1))           # 输出: True -- 非 0 转布尔是 True
print(bool(""))          # 输出: False -- 空字符串是 False
print(bool("a"))         # 输出: True -- 非空字符串是 True
print(round(3.7))        # 输出: 4 -- round() 四舍五入
print(round(3.14159, 2)) # 输出: 3.14 -- 保留 2 位小数的四舍五入

```

4. 运算符

```

# =====
# 算术运算符 -- 数学计算
# =====
a, b = 10, 3             # 先定义两个变量 a=10, b=3

print(a + b)             # 输出: 13 -- 加: 两数相加
print(a - b)             # 输出: 7 -- 减: 两数相减
print(a * b)             # 输出: 30 -- 乘: 两数相乘
print(a / b)             # 输出: 3.3333333333333335 -- 除: 结果永远是浮点数
print(a // b)            # 输出: 3 -- 整除: 只保留商的整数部分, 向下取整
print(a % b)             # 输出: 1 -- 取余 (取模): 10 ÷ 3 余 1
print(a ** b)            # 输出: 1000 -- 幂运算: 10 的 3 次方 = 1000

# 取余的实际应用
print(10 % 2 == 0)       # 输出: True -- 判断偶数: 能被 2 整除就是偶数
print(10 % 2 != 0)       # 输出: False -- 判断奇数: 不能被 2 整除就是奇数
print(2024 % 4 == 0)     # 输出: True -- 判断闰年的一部分

# =====
# 比较运算符 -- 结果都是布尔值 True/False
# =====
print(5 == 5)            # 输出: True -- 等于: 两个值相等
print(5 != 3)            # 输出: True -- 不等于: 两个值不相等
print(5 > 3)             # 输出: True -- 大于: 左边大于右边
print(5 < 3)             # 输出: False -- 小于: 左边小于右边

```

```

print(5 >= 5)          # 输出: True    — 大于等于: 左边大于或等于右边
print(5 <= 3)          # 输出: False   — 小于等于: 左边小于或等于右边

# 链式比较 (Python 特色)
x = 5
print(1 < x < 10)      # 输出: True    — 等价于 1 < x and x < 10
print(0 <= x <= 10)    # 输出: True    — 判断 x 是否在 0 到 10 之间

# =====
# 赋值运算符 — 给变量赋值
# =====
x = 5                  # 基础赋值: x = 5
print(x)              # 输出: 5

x += 3                # 等价于 x = x + 3 → 5 + 3 = 8
print(x)              # 输出: 8

x -= 2                # 等价于 x = x - 2 → 8 - 2 = 6
print(x)              # 输出: 6

x *= 2                # 等价于 x = x * 2 → 6 × 2 = 12
print(x)              # 输出: 12

x /= 3                # 等价于 x = x / 3 → 12 ÷ 3 = 4.0 (注意结果是
float)                # 输出: 4.0
print(x)

x //= 2               # 等价于 x = x // 2 → 4.0 // 2 = 2.0
print(x)              # 输出: 2.0

x **= 3               # 等价于 x = x ** 3 → 2.0³ = 8.0
print(x)              # 输出: 8.0

x %= 3                # 等价于 x = x % 3 → 8.0 % 3 = 2.0
print(x)              # 输出: 2.0

# =====
# 逻辑运算符 — and 与、or 或、not 非
# =====
print(True and False) # 输出: False   — 与: 两边都为 True 才是 True
print(True and True)  # 输出: True    — 与: 两边都 True → True
print(True or False)  # 输出: True    — 或: 只要一边是 True 就是 True
print(False or False) # 输出: False   — 或: 两边都 False → False
print(not True)        # 输出: False   — 非: 取反, True 变 False
print(not False)       # 输出: True    — 非: False 变 True

# 短路求值
print(0 and "hello")   # 输出: 0      — and 左边是假, 直接返回左边 (短路)
print(1 and "hello")   # 输出: hello   — and 左边是真, 返回右边
print(0 or "hello")    # 输出: hello   — or 左边是假, 返回右边
print(1 or "hello")    # 输出: 1       — or 左边是真, 直接返回左边 (短路)

# =====
# 成员运算符 — in 和 not in
# =====
print("a" in "abc")     # 输出: True    — "a" 在字符串 "abc" 中
print("d" not in "abc") # 输出: True    — "d" 不在字符串 "abc" 中

```

```

print(2 in [1, 2, 3])           # 输出: True    -- 2 在列表 [1,2,3] 中
print(5 not in [1, 2, 3])       # 输出: True    -- 5 不在列表中
print("name" in {"name": "Alice"}) # 输出: True    -- 字典中判断的是键 key

# =====
# 身份运算符 -- is 和 is not
# =====
a = [1, 2, 3]
b = a                           # b 和 a 指向同一个列表对象
print(a is b)                   # 输出: True    -- 同一个对象, 内存地址相同
print(a is not [1, 2, 3])       # 输出: True    -- 新建的列表是不同对象
print(a == [1, 2, 3])          # 输出: True    -- == 比较的是值, is 比较的是身份
                                (内存地址)

# 小整数池: -5 到 256 的整数会被缓存, is 判断为 True
print(100 is 100)               # 输出: True    (小整数池)
print(1000 is 1000)            # 输出: False   (大整数, 不同对象)

# =====
# 位运算符 -- 把数字转成二进制逐位运算
# =====
a, b = 5, 3                     # a = 5 = 101 (二进制), b = 3 = 011 (二进制)

print(a & b)                     # 输出: 1      -- 按位与: 101 & 011 = 001 = 1, 两位
都是1才得1
print(a | b)                     # 输出: 7      -- 按位或: 101 | 011 = 111 = 7, 有一
位是1就得1
print(a ^ b)                     # 输出: 6      -- 按位异或: 101 ^ 011 = 110 = 6, 不
同才得1
print(~a)                       # 输出: -6     -- 按位取反: ~101 = ...11111010 =
-6 (补码表示)
print(a << 1)                   # 输出: 10     -- 左移1位: 101 << 1 = 1010 = 10,
相当于 x2
print(a >> 1)                   # 输出: 2      -- 右移1位: 101 >> 1 = 10 = 2, 相当
于 ÷2 取整

# =====
# 运算符优先级 (从高到低, 记不住就加括号)
# =====
# () 括号最高 → ** 幂 → ~ 取反 → * / // % 乘除模 → + - 加减
# → << >> 移位 → & 按位与 → ^ 按位异或 → | 按位或
# → == != > < >= <= 比较 → not 非 → and 与 → or 或
result = 2 + 3 * 4               # 先乘后加 → 2 + 12 = 14
print(result)                   # 输出: 14
result = (2 + 3) * 4             # 有括号先算括号 → 5 × 4 = 20
print(result)                   # 输出: 20

```

5. 条件语句

```

# =====
# if-elif-else 多分支判断
# =====
score = 85
if score >= 90:
    print("优秀")

```

```

elif score >= 80:
    print("良好")                # 输出：良好 -- 85 在 80~89 之间
elif score >= 70:
    print("中等")
elif score >= 60:
    print("及格")
else:
    print("不及格")

# =====
# 嵌套 if -- if 里面再写 if
# =====
x = 10
if x > 0:                        # 先判断是否为正数
    if x % 2 == 0:              # 正数前提下再判断奇偶
        print("正偶数")        # 输出：正偶数
    else:
        print("正奇数")

# =====
# 三元表达式（条件表达式）-- 一行搞定简单的 if-else
# =====
age = 20
status = "成年" if age >= 18 else "未成年"
print(status)                   # 输出：成年

# 等价于：
if age >= 18:
    status = "成年"
else:
    status = "未成年"

# =====
# match-case 模式匹配（Python 3.10+ 新增）
# =====
day = 3
match day:
    case 1:
        print("星期一")
    case 2:
        print("星期二")
    case 3:
        print("星期三")        # 输出：星期三
    case _:
        print("未知")          # _ 表示通配，都不匹配时走这里

```

6. 循环语句

```

# =====
# for 循环 -- 遍历序列或指定次数
# =====
for i in range(5):              # range(5) 生成 0,1,2,3,4 共 5 个数
    print(i)                   # 依次输出：0 1 2 3 4

for i in range(1, 10, 2):      # range(起始, 结束, 步长) → 1,3,5,7,9

```

```

print(i)                                # 依次输出: 1 3 5 7 9

fruits = ["apple", "banana", "cherry"]
for fruit in fruits:                    # 直接遍历列表中的每个元素
    print(fruit)                        # 依次输出: apple banana cherry

# 带索引遍历 — enumerate()
for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")          # 输出: 0: apple 1: banana 2: cherry

# =====
# while 循环 — 条件满足就一直循环
# =====
count = 0
while count < 5:                        # 当 count < 5 时继续循环
    print(count)                        # 依次输出: 0 1 2 3 4
    count += 1                          # 每次循环 count 加 1, 防止死循环

# =====
# break — 跳出整个循环
# =====
for i in range(10):
    if i == 5:
        break                          # 当 i 等于 5 时, 直接跳出整个循环
    print(i)                            # 输出: 0 1 2 3 4 (到 5 就停了)

# =====
# continue — 跳过当前这一次, 继续下一次循环
# =====
for i in range(5):
    if i == 2:
        continue                       # 当 i 等于 2 时, 跳过这次, 继续下一次
    print(i)                            # 输出: 0 1 3 4 (跳过了 2)

# =====
# else 子句 — 循环正常结束 (没有被 break 中断) 才执行
# =====
for i in range(3):
    print(i)                            # 输出: 0 1 2
else:
    print("循环完成")                   # 输出: 循环完成 — 正常结束, 执行 else

# 如果循环被 break 打断, else 就不会执行
for i in range(3):
    if i == 1:
        break
else:
    print("循环完成")                   # 不会输出, 因为被 break 打断了

# =====
# 嵌套循环 — 循环里面还有循环
# =====
for i in range(3):                      # 外层循环执行 3 次
    for j in range(3):                  # 内层循环也执行 3 次
        print(f"{i},{j}", end=" ")     # 输出: 0,0 0,1 0,2 1,0 1,1 1,2 2,0 2,1 2,2
    print()                             # 每轮外层循环结束换一行

# =====

```

7. 序列类型

```
# =====
# 【字符串特性速览】
# ☒ 不可变 (Immutable)：创建后不能修改
# ☒ 有序：支持索引和切片
# ☒ 元素只能是字符
# ☒ 可重复：字符可以重复出现
# ☒ 可迭代：可以用 for 遍历
# ☒ 可作为字典的键
# =====

# =====
# 字符串创建
# =====

s1 = "hello"                # 双引号创建
s2 = 'world'                # 单引号创建（和双引号一样）
s3 = """多行
字符串"""                  # 三引号创建多行字符串

print(s1)                   # 输出：hello
print(s3)                   # 输出：多行 / 字符串

# =====
# 索引与切片 —— 序列类型通用操作
# =====

s = "Python"                # 索引：P=0, y=1, t=2, h=3, o=4, n=5
print(s[0])                  # 输出：P —— 取第 0 个字符（第一个）
print(s[-1])                 # 输出：n —— 取倒数第一个字符（最后一个）
print(s[1:4])                # 输出：yth —— 切片 [起始:结束)，左闭右开，取索引
1,2,3
print(s[:3])                 # 输出：Pyt —— 从头开始到索引 3 之前
print(s[3:])                 # 输出：hon —— 从索引 3 到末尾
print(s[::-2])               # 输出：Pto —— 步长为 2，每隔一个取一个
print(s[::-1])               # 输出：nohtyP —— 步长为 -1，从后往前取 = 反转

# =====
# 字符串常用方法 —— 因为不可变，所有方法都返回新字符串
# =====

s = " Hello world "

print(s.upper())              # 输出：" HELLO WORLD " —— 全部转大写
print(s.lower())              # 输出：" hello world " —— 全部转小写
print(s.strip())              # 输出："Hello world" —— 去除首尾空白字符
print(s.lstrip())             # 输出："Hello world " —— 只去除左侧空白
print(s.rstrip())             # 输出：" Hello world" —— 只去除右侧空白
```

```

s2 = "a,b,c,d"
print(s2.split(","))          # 输出: ['a', 'b', 'c', 'd'] -- 按逗号分割, 返回列表
print(",".join(['a','b','c'])) # 输出: "a,b,c" -- 用逗号把列表连起来

s3 = "hello"
print(s3.replace("l", "L"))   # 输出: "heLLo" -- 把所有 l 替换成 L
print(s3.find("e"))           # 输出: 1 -- 查找子串索引, 找不到返回 -1
print(s3.index("e"))          # 输出: 1 -- 查找子串索引, 找不到会报错 ValueError
print(s3.count("l"))          # 输出: 2 -- 统计子串出现的次数
print(s3.startswith("he"))    # 输出: True -- 是否以 "he" 开头
print(s3.endswith("lo"))      # 输出: True -- 是否以 "lo" 结尾

s4 = "123"
print(s4.isdigit())           # 输出: True -- 是否全是数字
s5 = "abc"
print(s5.isalpha())           # 输出: True -- 是否全是字母
s6 = "abc123"
print(s6.isalnum())           # 输出: True -- 是否只含字母或数字
s7 = "   "
print(s7.isspace())           # 输出: True -- 是否全是空白字符

# =====
# 字符串格式化 -- 三种方式
# =====
name, age = "Alice", 20

# 方式1: % 格式化 (老式, 不推荐)
print("My name is %s, I am %d years old." % (name, age))
# 输出: My name is Alice, I am 20 years old.
# %s 字符串 %d 整数 %f 浮点数

# 方式2: format() 方法
print("My name is {}, I am {} years old.".format(name, age))
# 输出: My name is Alice, I am 20 years old.

# 方式3: f-string (推荐, Python 3.6+)
print(f"My name is {name}, I am {age} years old.")
# 输出: My name is Alice, I am 20 years old.

print(f"Pi = {3.14159:.2f}")    # 输出: Pi = 3.14 -- 保留 2 位小数
print(f"{name:>10}")           # 输出: "      Alice" -- 右对齐, 总宽度 10
print(f"{age:0>5}")             # 输出: 00020 -- 左边补零, 宽度 5

# =====
# 字符串不可变性验证
# =====
s = "hello"
# s[0] = "H"                    # 错误! TypeError: 'str' object does not
#                               # support item assignment
# 要修改只能重新赋值:
s = "H" + s[1:]                # 拼接出一个新字符串
print(s)                       # 输出: Hello

```

7.2 列表 list

```
# =====
# 【列表特性速览】
# ☑ 可变 (Mutable)：创建后可以增删改元素
# ☑ 有序：支持索引和切片
# ☑ 可重复：允许元素重复出现
# ☑ 变长：可以动态增减元素
# ☑ 元素可以是任意类型（可混合类型）
# ☑ 可迭代：可以用 for 遍历
# ✗ 不能直接去重（需要手动或转 set）
# ✗ 不能作为字典的键（因为可变）
# =====

# =====
# 列表创建
# =====

lst = [1, 2, 3, 4, 5]          # 最常用的创建方式
mixed = [1, "hello", True, 3.14] # 可以混合不同类型的元素
empty = []                   # 空列表
lst2 = list("abc")           # 用 list() 函数从其他序列创建
print(lst)                   # 输出：[1, 2, 3, 4, 5]
print(mixed)                  # 输出：[1, 'hello', True, 3.14]
print(empty)                  # 输出：[]
print(lst2)                   # 输出：['a', 'b', 'c']

# =====
# 【查】索引与切片 —— 和字符串一样
# =====

print(lst[0])                 # 输出：1 —— 取第 0 个元素
print(lst[-1])                # 输出：5 —— 取最后一个元素
print(lst[1:4])               # 输出：[2, 3, 4] —— 切片，左闭右开
print(lst[::-1])              # 输出：[5, 4, 3, 2, 1] —— 反转

print(lst.index(3))            # 输出：2 —— 查找元素 3 的索引
print(lst.count(2))            # 输出：1 —— 统计元素 2 出现的次数
print(3 in lst)                # 输出：True —— 判断元素是否在列表中
print(len(lst))                # 输出：5 —— 列表长度（元素个数）
print(max(lst))                # 输出：5 —— 最大值
print(min(lst))                # 输出：1 —— 最小值
print(sum(lst))                # 输出：15 —— 求和（仅数字列表）

# =====
# 【增】添加元素
# =====

lst = [1, 2, 3]
print(lst)                    # 输出：[1, 2, 3]

lst.append(6)                  # 在末尾添加一个元素
print(lst)                     # 输出：[1, 2, 3, 6]

lst.insert(1, 10)              # 在索引 1 的位置插入 10，后面的元素后移
print(lst)                     # 输出：[1, 10, 2, 3, 6]

lst.extend([7, 8])             # 扩展列表，把另一个列表的元素逐个加进来
print(lst)                     # 输出：[1, 10, 2, 3, 6, 7, 8]
```

```

# =====
# 【删】删除元素
# =====

lst = [1, 2, 3, 4, 5, 3]
print(lst)                # 输出: [1, 2, 3, 4, 5, 3]

lst.remove(3)              # 删除第一个出现的 3 (按值删, 只删第一个)
print(lst)                # 输出: [1, 2, 4, 5, 3]

popped = lst.pop()        # 删除并返回最后一个元素
print(popped)             # 输出: 3 -- 被删除的元素
print(lst)                # 输出: [1, 2, 4, 5]

popped2 = lst.pop(1)      # 删除并返回索引 1 的元素
print(popped2)            # 输出: 2
print(lst)                # 输出: [1, 4, 5]

del lst[0]                # 用 del 删除指定索引的元素
print(lst)                # 输出: [4, 5]

lst.clear()               # 清空整个列表
print(lst)                # 输出: []

# =====
# 【改】修改元素
# =====

lst = [1, 2, 3, 4, 5]
lst[0] = 100              # 通过索引直接赋值修改
print(lst)                # 输出: [100, 2, 3, 4, 5]

lst[1:3] = [20, 30]       # 切片赋值, 同时修改多个元素
print(lst)                # 输出: [100, 20, 30, 4, 5]

# =====
# 【排序与反转】
# =====

lst = [3, 1, 4, 1, 5, 9, 2, 6]

lst.sort()                # 原地排序 (从小到大, 修改原列表)
print(lst)                # 输出: [1, 1, 2, 3, 4, 5, 6, 9]

lst.sort(reverse=True)    # 降序排序 (从大到小)
print(lst)                # 输出: [9, 6, 5, 4, 3, 2, 1, 1]

words = ["banana", "apple", "cherry"]
words.sort(key=len)       # 按自定义规则排序, 这里按字符串长度
print(words)              # 输出: ['apple', 'banana', 'cherry']

lst = [1, 2, 3]
lst.reverse()             # 原地反转 (修改原列表)
print(lst)                # 输出: [3, 2, 1]

# 注意区分: sorted() 返回新列表, 不改变原列表
lst = [3, 1, 2]
new_lst = sorted(lst)
print(lst)                # 输出: [3, 1, 2] -- 原列表不变
print(new_lst)            # 输出: [1, 2, 3] -- 新的排序后的列表

```

```

# =====
# 其他内置方法
# =====
lst = [1, 2, 3]
copy_lst = lst.copy()           # 浅拷贝（复制一份新的列表）
print(copy_lst)                 # 输出: [1, 2, 3]
print(lst is copy_lst)          # 输出: False — 是不同的列表对象

# =====
# 列表推导式 — 快速生成列表的语法糖
# =====
squares = [x**2 for x in range(10)]
print(squares)                  # 输出: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

evens = [x for x in range(20) if x % 2 == 0]
print(evens)                    # 输出: [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

matrix = [[i*j for j in range(1,4)] for i in range(1,4)]
print(matrix)                   # 输出: [[1,2,3], [2,4,6], [3,6,9]]

# =====
# 列表去重（需要手动，因为列表本身不去重）
# =====
lst = [1, 2, 2, 3, 3, 3]
unique = list(set(lst))         # 转 set 去重再转回 list, 但顺序可能变
print(unique)                   # 输出: [1, 2, 3]（顺序不一定）

# 去重并保持顺序
lst = [1, 2, 2, 3, 1]
seen = set()
result = []
for x in lst:
    if x not in seen:
        seen.add(x)
        result.append(x)
print(result)                   # 输出: [1, 2, 3] — 保持原顺序

```

7.3 元组 tuple

```

# =====
# 【元组特性速览】
# ☒ 不可变（Immutable）：创建后不能增删改元素
# ☒ 有序：支持索引和切片
# ☒ 可重复：允许元素重复出现
# ☒ 定长：创建后长度固定，不能改变
# ☒ 元素可以是任意类型（可混合类型）
# ☒ 可迭代：可以用 for 遍历
# ☒ 可作为字典的键（因为不可变）
# ☒ 不能去重（不可变，无法操作）
# =====

# =====
# 元组创建
# =====
t = (1, 2, 3)                  # 标准创建方式
print(t)                       # 输出: (1, 2, 3)

```

```

single = (1,)          # 单元素元组必须加逗号！否则不是元组
print(single)          # 输出：(1,)
print(type((1)))       # 输出：<class 'int'> -- 不加逗号就是整数
print(type((1,)))      # 输出：<class 'tuple'> -- 加了逗号才是元组

empty = ()             # 空元组
print(empty)           # 输出：()

t2 = tuple([1, 2, 3])  # 用 tuple() 从列表创建
print(t2)              # 输出：(1, 2, 3)

# 省略括号的写法（元组打包）
t3 = 1, 2, 3           # 不加括号也是元组
print(t3)              # 输出：(1, 2, 3)
print(type(t3))        # 输出：<class 'tuple'>

# =====
# 【查】索引与切片 -- 和列表完全一样
# =====
t = (1, 2, 3, 4, 5)
print(t[0])            # 输出：1 -- 取第 0 个元素
print(t[-1])           # 输出：5 -- 取最后一个元素
print(t[1:])           # 输出：(2, 3, 4, 5) -- 切片

print(t.index(3))      # 输出：2 -- 查找元素 3 的索引
print(t.count(2))      # 输出：1 -- 统计元素 2 出现次数
print(3 in t)          # 输出：True -- 判断是否存在
print(len(t))          # 输出：5 -- 元组长度
print(max(t))          # 输出：5 -- 最大值
print(min(t))          # 输出：1 -- 最小值
print(sum(t))          # 输出：15 -- 求和

# =====
# 【增】元组不可变，所以不能直接添加元素
# 但是可以通过拼接生成新元组（原元组不变）
# =====
t1 = (1, 2)
t2 = (3, 4)
t3 = t1 + t2           # 拼接生成新元组
print(t1)              # 输出：(1, 2) -- 原元组不变
print(t2)              # 输出：(3, 4)
print(t3)              # 输出：(1, 2, 3, 4) -- 新元组

t4 = t1 * 3             # 重复生成新元组
print(t4)              # 输出：(1, 2, 1, 2, 1, 2)

# =====
# 【删】元组不可变，不能删除单个元素
# 只能删除整个元组
# =====
t = (1, 2, 3)
# del t[0]             # 错误！TypeError：不支持删除单个元素
del t                  # 可以删除整个元组变量

# =====
# 【改】元组不可变，不能修改元素
# =====
t = (1, 2, 3)

```

```

# t[0] = 100                                # 错误! TypeError: 'tuple' object does not
support item assignment

# 但如果元组里包含可变对象（如列表），那个可变对象本身是可以改的
t = (1, [2, 3], 4)
t[1][0] = 200                                # 修改的是列表，不是元组本身
print(t)                                     # 输出: (1, [200, 3], 4) — 列表内容变了
# 注意: 元组的不可变指的是元组的元素指向不能变，但指向的对象如果是可变的，内容可以改

# =====
# 元组解包 — 把元组的元素分别赋给多个变量
# =====
a, b, c = (1, 2, 3)                          # 解包后 a=1, b=2, c=3
print(a, b, c)                              # 输出: 1 2 3

# 用 * 号接收多余的元素
first, *rest = (1, 2, 3, 4, 5)
print(first)                                # 输出: 1 — 第一个元素
print(rest)                                # 输出: [2, 3, 4, 5] — 剩余元素组成列表

*firsts, last = (1, 2, 3, 4, 5)
print(firsts)                              # 输出: [1, 2, 3, 4] — 前面的元素
print(last)                                # 输出: 5 — 最后一个元素

# 交换变量的本质就是元组打包和解包
a, b = 10, 20
a, b = b, a                                # 右边先打包成元组 (20, 10)，再解包给 a, b
print(a, b)                              # 输出: 20 10

# =====
# 元组的用途
# =====
# 1. 保护数据不被修改（作为常量集合）
DAYS_OF_WEEK = ("Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun")

# 2. 作为字典的键（列表不行，元组可以）
coordinates = {(0, 0): "原点", (1, 1): "第一象限"}
print(coordinates[(0, 0)])                # 输出: 原点

# 3. 函数返回多个值（本质是返回元组）
def get_point():
    return 10, 20                          # 返回的是元组 (10, 20)

x, y = get_point()
print(x, y)                              # 输出: 10 20

# =====
# 元组与列表互相转换
# =====
t = (1, 2, 3)
lst = list(t)                              # 元组转列表
print(lst)                                # 输出: [1, 2, 3]

t2 = tuple(lst)                            # 列表转元组
print(t2)                                # 输出: (1, 2, 3)

```

7.4 序列类型总结对比

7.4.1 特性对比表

特性	字符串 str	列表 list	元组 tuple
可变性	不可变	可变	不可变
有序性	有序	有序	有序
索引 / 切片	支持	支持	支持
元素类型	仅字符	任意类型	任意类型
可重复	是	是	是
可迭代	是	是	是
可作为字典键	是	否	是
定长 / 变长	定长	变长	定长
可去重	否	否（需手动）	否
性能特点	查询快	增删快	查询略快于列表
典型用途	文本处理	动态数据集合	固定数据记录、字典键

7.4.2 序列通用操作（共同点）

操作	写法示例	说明
索引访问	<code>seq[0]</code> 、 <code>seq[-1]</code>	正索引从 0 开始，负索引从末尾开始
切片	<code>seq[1:4]</code> 、 <code>seq[::-1]</code>	左闭右开，支持步长
长度	<code>len(seq)</code>	返回元素个数
成员判断	<code>"a" in "abc"</code>	<code>in</code> / <code>not in</code> 运算符
拼接	<code>"a" + "b"</code> 、 <code>[1] + [2]</code>	<code>+</code> 运算符，返回新序列
重复	<code>"a" * 3</code> 、 <code>[1] * 3</code>	<code>*</code> 运算符，重复 n 次
最值/求和	<code>min(seq)</code> 、 <code>max(seq)</code> 、 <code>sum(seq)</code>	元素需可比较/可运算
遍历	<code>for x in seq:</code>	都支持 for 循环迭代
统计次数	<code>seq.count(x)</code>	统计元素 x 出现的次数
查找索引	<code>seq.index(x)</code>	查找元素 x 的第一个索引

7.4.3 选择建议表

你的需求	推荐类型	理由
数据是文本、需要字符串操作	字符串 str	专门的文本处理方法（ <code>split</code> 、 <code>join</code> 、 <code>replace</code> 等）
数据需要增删改、动态变化	列表 list	可变 + 变长 + 丰富的内置方法
数据固定不变、需要保护	元组 tuple	不可变 + 可作字典键 + 性能略好

8. 映射与集合类型

8.1 字典 dict

```
# =====
# 【字典特性速览】
# ☑ 可变（Mutable）：可以增删改键值对
# ☑ 键不可重复：重复的键会覆盖前面的值（自动去重键）
# ☑ 键必须是不可变类型：str、int、float、tuple 等
# ☑ 值可以是任意类型
# ☑ 键值对映射：通过键快速查找值，O(1)
# ☑ 变长：可以动态增减键值对
# ☑ 可迭代：可以遍历键、值、键值对
# ⚠ Python 3.7+ 保持插入顺序，但逻辑上仍视为无序
# =====

# =====
# 字典创建
# =====
d = {"name": "Alice", "age": 20, "city": "Beijing"} # 最常用方式
print(d) # 输出：{'name': 'Alice', 'age': 20, 'city': 'Beijing'}

d2 = dict(name="Bob", age=25) # 用 dict() 函数创建
print(d2) # 输出：{'name': 'Bob', 'age': 25}

d3 = dict([("a", 1), ("b", 2)]) # 从键值对列表创建
print(d3) # 输出：{'a': 1, 'b': 2}

empty = {} # 空字典
print(empty) # 输出：{}

# =====
# 【查】访问元素
# =====
d = {"name": "Alice", "age": 20}

print(d["name"]) # 输出：Alice -- 通过键取值，键不存在会报错
# print(d["gender"]) # 错误！KeyError: 'gender'

print(d.get("age")) # 输出：20 -- get() 取值，键不存在返回 None
print(d.get("gender")) # 输出：None -- 不报错，返回 None
print(d.get("gender", "unknown")) # 输出：unknown -- 可以指定默认值
```

```

print("name" in d)           # 输出: True -- 判断键是否存在 (判断的是键, 不是值)
print("Alice" in d)         # 输出: False -- in 判断的是键, 不是值
print(len(d))               # 输出: 2 -- 键值对的数量

print(d.keys())             # 输出: dict_keys(['name', 'age']) -- 所有键
print(d.values())           # 输出: dict_values(['Alice', 20]) -- 所有值
print(d.items())            # 输出: dict_items([('name', 'Alice'), ('age', 20)]) -- 所有键值对

# =====
# 【增】添加键值对
# =====
d = {"name": "Alice"}
print(d)                    # 输出: {'name': 'Alice'}

d["age"] = 20                # 键不存在时就是添加
print(d)                    # 输出: {'name': 'Alice', 'age': 20}

d.update({"city": "Beijing", "gender": "female"}) # 批量添加/更新
print(d)                    # 输出: {'name': 'Alice', 'age': 20, 'city': 'Beijing', 'gender': 'female'}

d.setdefault("grade", 90)    # 如果键不存在就添加, 存在就不改变
print(d["grade"])            # 输出: 90
d.setdefault("grade", 100)   # 键已存在, 不会修改
print(d["grade"])            # 输出: 90 -- 还是原来的值

# =====
# 【删】删除键值对
# =====
d = {"name": "Alice", "age": 20, "city": "Beijing", "grade": 90}
print(d)                    # 输出: {'name': 'Alice', 'age': 20, 'city': 'Beijing', 'grade': 90}

del d["city"]                # 删除指定的键值对, 键不存在会报错
print(d)                    # 输出: {'name': 'Alice', 'age': 20, 'grade': 90}

age = d.pop("age")           # 删除并返回指定键的值
print(age)                  # 输出: 20
print(d)                    # 输出: {'name': 'Alice', 'grade': 90}

last_item = d.popitem()      # 删除并返回最后一个键值对 (LIFO, Python 3.7+)
print(last_item)            # 输出: ('grade', 90)
print(d)                    # 输出: {'name': 'Alice'}

d.clear()                   # 清空整个字典
print(d)                    # 输出: {}

# =====
# 【改】修改键值对
# =====
d = {"name": "Alice", "age": 20}
print(d)                    # 输出: {'name': 'Alice', 'age': 20}

d["age"] = 21                # 键已存在时就是修改

```

```

print(d)                # 输出: {'name': 'Alice', 'age': 21}

d.update({"age": 22, "city": "Shanghai"}) # update 可以同时修改和添加
print(d)                # 输出: {'name': 'Alice', 'age': 22, 'city':
                          'Shanghai'}

# =====
# 遍历字典的三种方式
# =====
d = {"name": "Alice", "age": 20, "city": "Beijing"}

# 方式1: 遍历键
for key in d:            # 直接遍历字典就是遍历键
    print(key)           # 输出: name age city

# 方式2: 遍历键值对 (最常用)
for key, value in d.items():
    print(f"{key}: {value}") # 输出: name: Alice age: 20 city: Beijing

# 方式3: 遍历值
for value in d.values():
    print(value)           # 输出: Alice 20 Beijing

# =====
# 字典推导式 -- 快速生成字典
# =====
squares = {x: x**2 for x in range(5)}
print(squares)            # 输出: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# 字典键值互换 (值必须是唯一的)
d = {"a": 1, "b": 2, "c": 3}
reverse_d = {v: k for k, v in d.items()}
print(reverse_d)          # 输出: {1: 'a', 2: 'b', 3: 'c'}

# =====
# 字典的去重特性
# =====
d = {"a": 1, "a": 2, "a": 3} # 键重复时, 后面的会覆盖前面的
print(d)                  # 输出: {'a': 3} -- 键自动去重

```

8.2 集合 set

```

# =====
# 【集合特性速览】
# ☒ 可变 (Mutable): 可以增删元素
# ☒ 自动去重: 元素不可重复, 自动去重
# ☒ 元素必须是不可变类型: str、int、float、tuple 等
# ☒ 无序: 元素没有固定顺序, 不支持索引和切片
# ☒ 变长: 可以动态增减元素
# ☒ 可迭代: 可以用 for 遍历
# ☒ 支持数学集合运算: 并集、交集、差集、对称差集
# ☒ 查找效率高: O(1)
# ☒ 不能作为字典的键 (因为可变)
# =====

# =====
# 集合创建

```

```

# =====
s = {1, 2, 3, 3, 3}          # 自动去重，重复元素只保留一个
print(s)                   # 输出: {1, 2, 3}

s2 = set([1, 2, 2, 3, 3, 3]) # 用 set() 从列表创建，自动去重
print(s2)                  # 输出: {1, 2, 3}

s3 = set("hello")           # 从字符串创建
print(s3)                  # 输出: {'h', 'e', 'l', 'o'} -- 自动去重，顺序不定

empty = set()              # 空集合必须用 set()，不能用 {}
print(empty)               # 输出: set()
print(type({}))            # 输出: <class 'dict'> -- {} 是空字典!

# =====
# 【查】集合元素的访问与判断
# =====
s = {1, 2, 3}
# print(s[0])              # 错误! 集合不支持索引访问 (因为无序)

print(2 in s)              # 输出: True -- 判断元素是否在集合中
print(4 in s)              # 输出: False
print(len(s))              # 输出: 3 -- 元素个数

for x in s:                # 可以用 for 遍历，但顺序不固定
    print(x)               # 输出: 1 2 3 (顺序不一定)

# =====
# 【增】添加元素
# =====
s = {1, 2, 3}
print(s)                   # 输出: {1, 2, 3}

s.add(4)                   # 添加一个元素
print(s)                   # 输出: {1, 2, 3, 4}

s.add(4)                   # 添加已存在的元素，不会有变化 (自动去重)
print(s)                   # 输出: {1, 2, 3, 4}

s.update([5, 6, 7])        # 批量添加多个元素
print(s)                   # 输出: {1, 2, 3, 4, 5, 6, 7}

# =====
# 【删】删除元素
# =====
s = {1, 2, 3, 4, 5}
print(s)                   # 输出: {1, 2, 3, 4, 5}

s.remove(3)                # 删除指定元素，元素不存在会报错 KeyError
print(s)                   # 输出: {1, 2, 4, 5}

# s.remove(10)             # 错误! KeyError: 10

s.discard(5)               # 删除指定元素，元素不存在也不报错
print(s)                   # 输出: {1, 2, 4}

s.discard(10)              # 不存在也不报错，什么也不做

```

```

print(s)                # 输出: {1, 2, 4}

popped = s.pop()        # 随机删除并返回一个元素 (因为无序, 不知道删哪个)
print(popped)           # 输出: 1 (不一定)
print(s)                # 输出: {2, 4} (不一定)

s.clear()               # 清空整个集合
print(s)                # 输出: set()

# =====
# 【改】集合没有直接修改元素的方法
# 因为元素是不可变类型, 要"改"只能先删后加
# =====
s = {1, 2, 3}
s.remove(2)             # 先删
s.add(20)               # 再加
print(s)                # 输出: {1, 3, 20} -- 相当于把 2 改成 20

# =====
# 数学集合运算 -- 集合的核心功能
# =====
a = {1, 2, 3}
b = {3, 4, 5}

print(a | b)            # 输出: {1, 2, 3, 4, 5} -- 并集: 两个集合所有元素
print(a.union(b))       # 输出: {1, 2, 3, 4, 5} -- 方法名写法

print(a & b)            # 输出: {3} -- 交集: 两个集合都有的元素
print(a.intersection(b)) # 输出: {3} -- 方法名写法

print(a - b)            # 输出: {1, 2} -- 差集: a 有但 b 没有的
print(a.difference(b))  # 输出: {1, 2} -- 方法名写法

print(a ^ b)            # 输出: {1, 2, 4, 5} -- 对称差集: 不同时存在的元素
print(a.symmetric_difference(b)) # 输出: {1, 2, 4, 5} -- 方法名写法

# 子集与超集判断
a = {1, 2}
b = {1, 2, 3}
print(a.issubset(b))    # 输出: True -- a 是 b 的子集
print(b.issuperset(a))  # 输出: True -- b 是 a 的超集
print(a.isdisjoint({4, 5})) # 输出: True -- 两个集合没有交集 (互不相交)

# =====
# 集合推导式
# =====
evens = {x for x in range(10) if x % 2 == 0}
print(evens)            # 输出: {0, 2, 4, 6, 8}

# =====
# 集合的典型应用
# =====
# 应用1: 列表去重
lst = [1, 2, 2, 3, 3, 3]
unique = list(set(lst)) # 转集合去重, 再转回列表
print(unique)           # 输出: [1, 2, 3] (顺序不一定)

# 应用2: 找共同元素 (交集)

```

```
list1 = [1, 2, 3, 4, 5]
list2 = [4, 5, 6, 7, 8]
common = set(list1) & set(list2)
print(common)                                # 输出: {4, 5}

# 应用3: 快速判断元素是否存在 (O(1) 效率)
s = set(range(10000))
print(9999 in s)                            # 输出: True — 比列表快很多
```

8.3 映射与集合总结对比

8.3.1 特性对比表

特性	字典 dict	集合 set
可变性	可变	可变
有序性	无序 (3.7+ 保插入序)	无序
索引 / 切片	不支持	不支持
元素重复	键不可重复, 值可重复	元素不可重复
元素类型要求	键必须不可变, 值任意	元素必须不可变
查找效率	O(1) (按键查)	O(1)
存储形式	键值对	单元素
定长 / 变长	变长	变长
可去重	键自动去重	元素自动去重
可作为字典键	否	否
典型用途	键值对映射、计数、查找	去重、集合运算

8.3.2 共同点对照表

共同点	字典 dict	集合 set	说明
字面量符号	<code>{}</code>	<code>{}</code>	空集合必须用 <code>set()</code> ， <code>{}</code> 是空字典
元素不可变要求	键必须不可变	元素必须不可变	都需要可哈希 (hashable)
<code>len()</code>	<code>len(d)</code>	<code>len(s)</code>	都支持长度函数
<code>in</code> 运算符	<code>key in d</code>	<code>x in s</code>	都支持成员判断，效率 $O(1)$
<code>for</code> 遍历	<code>for k in d:</code>	<code>for x in s:</code>	都支持迭代
推导式	字典推导式	集合推导式	都有推导式语法
底层实现	哈希表	哈希表	都基于哈希表，查找快
可变性	可变	可变	都支持增删操作
变长	变长	变长	都可以动态增减元素
自动去重	键自动去重	元素自动去重	重复会被覆盖/忽略

8.3.3 选择建议表

你的需求	推荐类型	理由
需要存储键值对关系、通过键快速查值	字典 dict	键值映射 + $O(1)$ 查找
需要去重、做集合运算（交并补差）	集合 set	自动去重 + 数学集合运算
需要有序序列、按索引访问	列表 list	有序 + 可变 + 索引

9. 四大容器类型终极对比（列表/元组/集合/字典）

9.1 完整特性对比表

特性	列表 list	元组 tuple	集合 set	字典 dict
字面量符号	[]	()	{ }	{ }
空创建方式	[]	()	set()	{ }
可变性	可变	不可变	可变	可变
有序性	有序	有序	无序	无序 (3.7+ 保序)
索引 / 切片	支持	支持	不支持	不支持
元素重复	允许重复	允许重复	自动去重	键不可重复, 值可重复
元素类型要求	任意类型	任意类型	必须不可变	键必须不可变, 值任意
定长 / 变长	变长	定长	变长	变长
可去重	否 (需手动)	否	是 (自动)	键自动去重
可作为字典键	否	是	否	否
查找效率	O(n)	O(n)	O(1)	O(1) (按键查)
增删效率	末尾快 O(1)	不支持	快 O(1)	快 O(1)
可迭代	是	是	是	是 (遍历键/值/对)
推导式	列表推导式	—	集合推导式	字典推导式
典型用途	动态数据序列	固定数据记录	去重、集合运算	键值对映射、查找

9.2 CRUD 操作速查表

操作	列表 list	元组 tuple	集合 set	字典 dict
增	<code>append()</code> 末尾添加	不可增（只能拼接生成新元组）	<code>add()</code> 添加单个	<code>d[key] = value</code> 赋值
	<code>insert()</code> 指定位置插入		<code>update()</code> 批量添加	<code>update()</code> 批量更新/添加
	<code>extend()</code> 扩展列表			<code>setdefault()</code> 不存在才添加
删	<code>remove()</code> 按值删	不可删（只能删整个元组）	<code>remove()</code> 删除（不存在报错）	<code>del d[key]</code> 删除键值对
	<code>pop()</code> 按索引删		<code>discard()</code> 删除（不存在不报错）	<code>pop()</code> 删除并返回值
	<code>del</code> 按索引删		<code>pop()</code> 随机删除并返回	<code>popitem()</code> 删除最后一对
	<code>clear()</code> 清空		<code>clear()</code> 清空	<code>clear()</code> 清空
改	<code>lst[i] = x</code> 索引赋值	不可改	先删后加（元素不可变）	<code>d[key] = value</code> 键赋值
	切片赋值			<code>update()</code> 批量修改
查	<code>lst[i]</code> 索引访问	<code>t[i]</code> 索引访问	<code>in</code> 判断是否存在	<code>d[key]</code> 按键取值
	<code>index()</code> 查找索引	<code>index()</code> 查找索引	<code>for</code> 遍历	<code>get()</code> 取值（有默认值）
	<code>count()</code> 统计次数	<code>count()</code> 统计次数		<code>keys()</code> 所有键
	<code>in</code> 判断存在	<code>in</code> 判断存在		<code>values()</code> 所有值
	<code>len()</code> 长度	<code>len()</code> 长度	<code>len()</code> 长度	<code>items()</code> 所有键值对
				<code>in</code> 判断键存在

9.3 类型转换表

源类型	目标类型	写法	结果示例	备注
列表 [1,2,3]	元组	<code>tuple(lst)</code>	<code>(1, 2, 3)</code>	顺序不变
元组 (1,2,3)	列表	<code>list(t)</code>	<code>[1, 2, 3]</code>	顺序不变
列表 [1,2,2,3]	集合	<code>set(lst)</code>	<code>{1, 2, 3}</code>	自动去重 ，顺序不定
元组 (1,2,2,3)	集合	<code>set(t)</code>	<code>{1, 2, 3}</code>	自动去重 ，顺序不定
集合 {1,2,3}	列表	<code>list(s)</code>	<code>[1, 2, 3]</code>	顺序不定
集合 {1,2,3}	元组	<code>tuple(s)</code>	<code>(1, 2, 3)</code>	顺序不定
字典 {'a':1}	列表（键）	<code>list(d.keys())</code>	<code>['a']</code>	取所有键
字典 {'a':1}	列表（值）	<code>list(d.values())</code>	<code>[1]</code>	取所有值
字典 {'a':1}	列表（键值对）	<code>list(d.items())</code>	<code>[('a', 1)]</code>	取所有键值对

9.4 去重能力对比

容器类型	是否自动去重	去重方式	是否保留顺序	时间复杂度
列表 list	✗ 否	手动转 set 再转回 list	✗ 否（转 set 后顺序丢失）	O(n)
元组 tuple	✗ 否	不可变，无法操作	—	—
集合 set	☑ 是	天生去重	✗ 否（无序）	O(n)
字典 dict	☑ 键去重	键自动去重，值可重复	☑ 是（3.7+ 保插入序）	O(n)

9.5 选择容器决策表

你的需求	推荐容器	理由
需要按序号（索引）访问，且数据会增删改	列表 list	有序 + 可变 + 变长
需要按序号访问，但数据固定不变	元组 tuple	有序 + 不可变 + 可作字典键
需要快速查找、需要去重、做集合运算	集合 set	O(1) 查找 + 自动去重 + 交并补差
需要按键快速查值、存储键值对映射关系	字典 dict	O(1) 按键查 + 键去重 + 保序

10. 函数

```
# =====
# 定义函数 — def 关键字
# =====
def greet(name):
    """文档字符串 docstring — 说明函数的作用"""
    return f"Hello, {name}!"

print(greet("Alice"))          # 输出: Hello, Alice! — 调用函数

# =====
# 默认参数 — 调用时可以不传, 用默认值
# =====
def power(base, exponent=2):
    return base ** exponent

print(power(3))                # 输出: 9 — 不传 exponent, 用默认值 2, 即 32
print(power(2, 3))             # 输出: 8 — 传了 3, 即 23

# =====
# 位置参数 vs 关键字参数
# =====
def add(a, b):
    return a + b

print(add(3, 5))               # 输出: 8 — 位置参数: 按顺序传
print(add(a=3, b=5))           # 输出: 8 — 关键字参数: 按名字传
print(add(3, b=5))             # 输出: 8 — 混合使用, 位置参数必须在前面

# =====
# 可变参数 *args — 接收任意多个位置参数, 打包成元组
# =====
def sum_all(*args):
    print(args)                # args 是一个元组
    return sum(args)

print(sum_all(1, 2, 3, 4))     # 输出: (1, 2, 3, 4) → 10

# =====
# 关键字可变参数 **kwargs — 接收任意多个关键字参数, 打包成字典
# =====
def print_info(**kwargs):
    print(kwargs)              # kwargs 是一个字典
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_info(name="Alice", age=20) # 输出: {'name': 'Alice', 'age': 20} / name: Alice / age: 20

# =====
# 混合参数的顺序 (必须遵守)
# 位置参数 → 默认参数 → *args → **kwargs
# =====
def func(a, b=2, *args, **kwargs):
    print(f"a={a}, b={b}, args={args}, kwargs={kwargs}")
```

```

func(1, 3, 4, 5, x=10, y=20)
# 输出: a=1, b=3, args=(4, 5), kwargs={'x': 10, 'y': 20}

# =====
# Lambda 表达式 -- 匿名函数, 一行搞定
# =====
square = lambda x: x ** 2          # 定义匿名函数并赋值给变量
print(square(5))                  # 输出: 25

# 常用场景: 作为参数传给高阶函数
nums = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x ** 2, nums))
print(squares)                    # 输出: [1, 4, 9, 16, 25]

# =====
# 递归函数 -- 函数自己调用自己
# =====
def factorial(n):
    if n <= 1:                     # 递归终止条件 (必须有, 否则死循环)
        return 1
    return n * factorial(n - 1)    # 递归调用

print(factorial(5))               # 输出: 120 -- 5! = 5×4×3×2×1 = 120

# =====
# 变量作用域
# =====
global_var = 10                   # 全局变量: 在函数外定义, 整个文件都能用

def func():
    local_var = 5                 # 局部变量: 只在函数内部有效
    global global_var             # 声明要修改全局变量
    global_var = 20               # 修改全局变量
    print(local_var)              # 输出: 5
    print(global_var)             # 输出: 20

func()
print(global_var)                 # 输出: 20 -- 全局变量已被修改

# =====
# 内置高阶函数
# =====
nums = [1, 2, 3, 4, 5]

# map: 把每个元素都做一次变换
print(list(map(str, nums)))      # 输出: ['1', '2', '3', '4', '5'] -- 都转成字符串

# filter: 过滤出满足条件的元素
print(list(filter(lambda x: x > 2, nums))) # 输出: [3, 4, 5] -- 大于 2 的

# zip: 把两个序列按位置配对
print(list(zip([1,2], ['a','b']))) # 输出: [(1, 'a'), (2, 'b')]

# enumerate: 带索引遍历
print(list(enumerate(['a','b']))) # 输出: [(0, 'a'), (1, 'b')]

# sorted: 排序 (返回新列表, 不修改原列表)

```

```
print(sorted([3, 1, 2]))          # 输出: [1, 2, 3]
print(sorted([3, 1, 2], reverse=True)) # 输出: [3, 2, 1] -- 降序
```

11. 面向对象编程

```
# =====
# 类与对象 -- class 关键字定义类
# =====

class Dog:
    species = "Canis familiaris"    # 类属性: 所有实例共享

    def __init__(self, name, age):  # 构造方法: 创建对象时自动调用
        self.name = name           # 实例属性: 每个对象自己的
        self.age = age

    def bark(self):                 # 实例方法: 第一个参数必须是 self
        return f"{self.name} says woof!"

    def __str__(self):              # 字符串表示: print() 时调用
        return f"{self.name} is {self.age} years old"

    def __repr__(self):             # 官方表示: repr() 或交互模式下调用
        return f"Dog('{self.name}', {self.age})"

my_dog = Dog("Buddy", 3)           # 创建对象 (实例化)
print(my_dog.name)                 # 输出: Buddy -- 访问属性
print(my_dog.bark())               # 输出: Buddy says woof! -- 调用方法
print(my_dog)                     # 输出: Buddy is 3 years old -- 调用 __str__

# =====
# 类方法和静态方法
# =====

class Person:
    count = 0                       # 类属性: 统计创建了多少个对象

    def __init__(self, name):
        self.name = name
        Person.count += 1          # 每创建一个对象, 计数加 1

    @classmethod
    def get_count(cls):             # 类方法装饰器, 第一个参数是 cls (类本身)
        return cls.count           # 访问类属性

    @staticmethod
    def is_adult(age):              # 静态方法装饰器, 不需要 self 或 cls
        return age >= 18           # 就是一个普通函数, 只是放在类里

print(Person.get_count())           # 输出: 0 -- 通过类调用类方法
p1 = Person("Alice")
p2 = Person("Bob")
print(Person.get_count())           # 输出: 2 -- 创建了 2 个对象
print(Person.is_adult(20))          # 输出: True -- 调用静态方法

# =====
# 继承 -- 子类继承父类的属性和方法
```

```

# =====
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        raise NotImplementedError("子类必须实现这个方法")

class Cat(Antimal):
    # Cat 继承 Animal
    # 重写父类的方法
    def speak(self):
        return f"{self.name} says meow!"

class Dog(Antimal):
    # Dog 也继承 Animal
    def speak(self):
        return f"{self.name} says woof!"

cat = Cat("Kitty")
print(cat.speak())
# 输出: Kitty says meow!

dog = Dog("Buddy")
print(dog.speak())
# 输出: Buddy says woof!

# =====
# 多继承 -- 一个类可以继承多个父类
# =====
class A:
    def method_a(self):
        return "A"

class B:
    def method_b(self):
        return "B"

class C(A, B):
    # C 同时继承 A 和 B
    pass

c = C()
print(c.method_a())
# 输出: A -- 继承自 A
print(c.method_b())
# 输出: B -- 继承自 B

# =====
# 方法重写 + super() 调用父类方法
# =====
class Dog(Antimal):
    def __init__(self, name, age):
        super().__init__(name)
        # 调用父类的构造方法
        self.age = age
        # 子类自己新增的属性

    def speak(self):
        return f"{self.name} (age {self.age}) says woof!"

dog = Dog("Buddy", 3)
print(dog.speak())
# 输出: Buddy (age 3) says woof!

# =====
# 封装 -- 私有属性 (双下划线开头)
# =====
class BankAccount:

```

```

def __init__(self, balance):
    self.__balance = balance    # 私有属性：双下划线开头，外部不能直接访问

@property                      # 属性装饰器：把方法变成属性来访问
def balance(self):
    return self.__balance      # 读取私有属性

@balance.setter                # setter 装饰器：控制修改
def balance(self, value):
    if value >= 0:              # 加了校验逻辑
        self.__balance = value

def deposit(self, amount):
    if amount > 0:
        self.__balance += amount

account = BankAccount(100)
account.deposit(50)
print(account.balance)         # 输出：150 —— 通过 property 访问
# print(account.__balance)     # 错误！AttributeError：不能直接访问私有属性

# =====
# 常用魔术方法（双下划线方法）
# =====
# __init__()      构造方法，创建对象时调用
# __str__()       str() 和 print() 调用，返回用户友好的字符串
# __repr__()      repr() 调用，返回开发者友好的表示
# __len__()       len() 调用
# __eq__()        == 运算符
# __lt__()        < 运算符
# __add__()       + 运算符
# __getitem__()   [] 索引访问
# __iter__()      迭代器支持
# __contains__()  in 运算符

```

12. 文件读写

```

# =====
# 打开文件的模式 mode
# =====
# 'r'   只读（默认），文件不存在报错
# 'w'   只写，文件不存在就创建，存在就覆盖（清空重写）
# 'a'   追加，文件不存在就创建，存在就往末尾加
# 'x'   独占创建，文件已存在就报错
# 'b'   二进制模式（配合 r/w/a 使用，如 'rb' 'wb'）
# '+'   读写模式（配合 r/w/a 使用，如 'r+' 'w+'）

# =====
# with 语句 —— 自动关闭文件（推荐用法）
# =====
# with 语句结束后会自动调用 f.close()，即使发生异常也会关闭

# =====
# 读取文件
# =====

```

```

# 方式1: 读取整个文件
with open("data.txt", "r", encoding="utf-8") as f:
    content = f.read()          # 一次性读取全部内容
    print(content)

# 方式2: 逐行读取 (最常用, 适合大文件)
with open("data.txt", "r", encoding="utf-8") as f:
    for line in f:              # 逐行迭代, 每次读一行
        print(line.strip())     # strip() 去掉换行符和首尾空白

# 方式3: 读取所有行到列表
with open("data.txt", "r", encoding="utf-8") as f:
    lines = f.readlines()       # 每行作为一个元素, 返回列表
    print(lines)

# =====
# 写入文件
# =====
# 写入模式 'w' -- 会覆盖原有内容
with open("output.txt", "w", encoding="utf-8") as f:
    f.write("Hello, world!\n")   # 写一行 (需要自己加 \n)
    f.writelines(["Line 1\n", "Line 2\n"]) # 写多行 (列表中的字符串)

# 追加模式 'a' -- 往末尾添加, 不覆盖
with open("log.txt", "a", encoding="utf-8") as f:
    f.write("New log entry\n")   # 追加一行

# =====
# 文件指针操作
# =====
with open("data.txt", "r", encoding="utf-8") as f:
    print(f.tell())              # 输出: 0 -- 当前指针位置 (字节数)
    f.seek(10)                  # 移动指针到第 10 字节处
    print(f.tell())              # 输出: 10
    f.seek(0)                   # 移动到文件开头

# =====
# CSV 文件读写 -- 用 csv 模块
# =====
import csv

# 读取 CSV
with open("data.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f)       # 创建 reader 对象
    for row in reader:           # 逐行读取, 每行是一个列表
        print(row)

# 写入 CSV
with open("output.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)       # 创建 writer 对象
    writer.writerow(["Name", "Age"]) # 写表头
    writer.writerow(["Alice", 20])  # 写一行数据
    writer.writerows([["Bob", 25], ["Charlie", 30]]) # 写多行

# =====
# JSON 文件读写 -- 用 json 模块
# =====
import json

```

```

data = {"name": "Alice", "age": 20, "hobbies": ["reading", "music"]}

# 写入 JSON
with open("data.json", "w", encoding="utf-8") as f:
    json.dump(data, f, indent=4, ensure_ascii=False)
    # indent=4 缩进 4 空格, ensure_ascii=False 保留中文

# 读取 JSON
with open("data.json", "r", encoding="utf-8") as f:
    data = json.load(f)          # 读取并解析成 Python 对象
    print(data)                 # 输出: {'name': 'Alice', 'age': 20, ...}

```

13. 异常处理

```

# =====
# 基本异常处理 try-except
# =====
try:
    result = 10 / 0          # 这里会发生 ZeroDivisionError
except ZeroDivisionError:
    print("不能除以零!")    # 捕获到异常后执行这里
except Exception as e:     # 捕获其他所有异常
    print(f"发生错误: {e}")
else:
    print("没有异常时执行") # try 中没有异常才执行
finally:
    print("无论有无异常都执行") # 不管有没有异常都会执行 (清理工作)

# =====
# 捕获多个异常类型
# =====
try:
    x = int("abc")          # 会发生 ValueError
except (ValueError, TypeError): # 可以一次捕获多种异常
    print("转换失败")

# =====
# 获取异常信息
# =====
try:
    lst = [1, 2, 3]
    print(lst[10])          # 会发生 IndexError
except IndexError as e:    # as e 把异常对象赋值给 e
    print(f"错误类型: {type(e).__name__}") # 输出: 错误类型: IndexError
    print(f"错误信息: {e}")          # 输出: 错误信息: list index out of range

# =====
# 主动抛出异常 -- raise
# =====
def divide(a, b):
    if b == 0:
        raise ZeroDivisionError("除数不能为零") # 主动抛出异常
    return a / b

```

```

try:
    divide(10, 0)
except ZeroDivisionError as e:
    print(e)                # 输出：除数不能为零

# =====
# 自定义异常 -- 继承 Exception
# =====
class ValidationError(Exception):
    """自定义验证错误基类"""
    pass

class AgeError(ValidationError):
    """年龄相关的错误"""
    pass

def validate_age(age):
    if age < 0:
        raise AgeError("年龄不能为负数")
    if age > 150:
        raise AgeError("年龄不能超过150")
    return age

try:
    validate_age(200)
except AgeError as e:
    print(f"年龄错误: {e}")    # 输出：年龄错误：年龄不能超过150

# =====
# 异常链 -- raise ... from
# =====
try:
    int("abc")
except ValueError as e:
    raise RuntimeError("转换失败") from e # 保留原始异常信息

# =====
# 常见内置异常（必须记住）
# =====
# SyntaxError          语法错误（代码写错了）
# NameError           变量未定义就使用
# TypeError           类型错误（比如字符串加数字）
# ValueError          值错误（类型对但值不对，比如 int("abc")）
# IndexError          索引越界（列表/字符串下标超范围）
# KeyError            字典键不存在
# AttributeError      属性/方法不存在
# FileNotFoundError   文件不存在
# ZeroDivisionError   除以零
# ImportError         导入模块失败
# ModuleNotFoundError  模块不存在
# RuntimeError        运行时错误
# NotImplementedError  方法未实现
# StopIteration       迭代器结束
# AssertionError      断言失败

```

